

The three questions

Track A (inbound leads) is the submission. Track C (AEO) is the second half. Both are here because they are one engine pointed at two problems: a config file holds the contract, deterministic rules decide, the model does only what rules cannot, and every unmeasured cell is declared rather than defaulted to zero.

1. How I'd measure impact

Track C — headline: how many of ten buyer-intent queries we appear in. A count, not a percentage: "30%" reads identically whether it is 3 of 10 measured or 3 of 10 with four cells nobody looked at. Today it is **0 of 10**. Target 10 of 10.

- **Baseline first: the same ten queries daily for two weeks, to establish the noise band.** Assistant answers are non-deterministic — ask twice, the competitor set moves, and movement smaller than that spread is not a result. The tool already repeats a subset three times per run and reports how many cells contradicted themselves.
- **Leading indicator: citation share. Lagging: AI-referred sessions,** which stay near zero long after citations move — judging the programme on it in quarter one kills it.
- The 0–100 score never blends out of sight: four components each keep their own points, and one we could not measure is **excluded from the denominator** rather than scored zero (today 0 of 90 — nothing named us, so there is no position to hold).

Track A — headline: speed-to-lead against the SLA. What share of hot leads were contacted inside five minutes. Biggest single lever on inbound conversion.

- **Drafts citing zero facts.** The model declares which record fields each message leaned on; a draft citing nothing is a mail-merge in costume. This is how "personalised" stays checkable rather than taken on trust.
- **needs review rate** and **reconciliation mismatches** — leads that could not be looked up, or whose profile did not match the email domain. Nobody goes looking for the leads that never arrived, so these rot quietly.
- **The honest counterfactual is a holdout:** route a random 10% the old manual way and compare meeting-booked rate, or any lift gets credited to the scoring model when someone merely replied faster.

2. How I'd scale it

What breaks at 10x is the calls, not the analysis. Both tracks are serial HTTP; neither SQLite nor the arithmetic is near a limit.

- **Track C (100 queries x 4 engines x 5 repeats daily):** worker pool with a per-engine rate limiter; cache answers by content hash so a re-run does not re-buy an unchanged answer; Postgres (same schema, different driver). Then, in order: **close the two seam engines** (a credential gap, not a design gap — each is one function with the same signature), an **LLM extraction pass for competitor discovery** (today it only finds rivals cited from their own domains), **per-market query sets for DE/NL/PL** since AI answers are locale-sensitive and Europe is the expansion, and **alerting on movement outside the noise band** rather than a dashboard someone must remember to open.
- **Track A (500 leads/week):** at the measured \$0.06 a lead that is ~\$30/week — **cost is not the constraint, accuracy is.** No single B2B provider exceeds ~70% match rate on a real inbound list, so one provider means one lead in three arrives blank; chaining three to five lifts it to 85–95%. Then queue the enrichment (a form submission currently blocks on three HTTP calls) and **write back to the CRM** — scoring is worthless if sales must open a separate dashboard to see it.

3. One tradeoff, and what production would do differently

Two of four assistant surfaces are unmeasured. ChatGPT and Perplexity have no key on this account, so the count reflects Claude and Google AI Overviews only.

I made that a **declared seam** rather than quietly reporting a smaller matrix: an engine with no credential is stored **unmeasured**, excluded from every rate, and drawn hatched, because "we did not look" and "we looked and found nothing" are different findings. **Production closes it with two API keys** — but not with the OpenAI API alone: querying the API is not querying ChatGPT, and a cell reading "ChatGPT: cited" sourced from a different surface is a worse lie than an honest blank.

Apify is a convenience, not a recommendation. Every scrape here runs through it because I already had an account and could be live the same afternoon. It is a **marketplace of third-party scrapers, not a data vendor**, and the build showed what that costs: two actors doing the same job returned incompatible shapes, so a working scrape silently produced a title parsed from marketing copy, and free-plan gating changed the failure mode twice mid-project. Production buys the data, not the scraper — **Cognism** for European firmographics with lawful basis and DNC screening, **Clay** to orchestrate past three providers. On the AEO side Apify is a workaround for a surface with **no official API** (Serper returns no AI Overview field — tested), so production uses a SERP provider with native AI Overview support, or buys the monitoring outright: **Otterly ~\$29/month, Peec ~€89**. If Nebius only wants monitoring, buy it — what is worth keeping from this build is the source gap and the recommendation layer, the part none of them do.

Smaller ones: one locale (English); headcount growth needs a second sighting, so it is worth nothing on day one and everything by month three; competitor discovery misses rivals discussed but never linked.

Why building beat diagramming. Every adapter here was written from documentation and had never been executed. Running it found five faults and **four fail silently** — plausible output, no error. A wrong input key made a paid scraper return zero items, indistinguishable from "this person has no profile." None of them are visible in a workflow diagram.